

Sprawozdanie z zadania rekrutacyjnego

Prognoza sprzedaży w sztukach

Autor: Michał Pragacz

Data: 07.09.2026

Zadania:

- Wczytaj dane i zgreguj dzienną sprzedaż (Units Sold) sumując po sklepach i produktach.
- Zbadaj dane, wybierz czynniki zewnętrzne wpływające na sprzedaż i opisz w README swój wybór.
- Odłóż ostatnie 4 tygodnie (28 dni) jako zbiór walidacyjny; model trenuj wyłącznie na wcześniejszych danych.
- Wybierz model do prognozy szeregów czasowych i wygeneruj prognozę na 4 tygodnie do przodu.
- Policz metryki *MAE*, *RMSE*, *MAPE* oraz *bias* i krótko opisz wyniki.
- Zbuduj aplikację webową (np. Streamlit, Plotly Dash): wykres z prognozą oraz tabela metryk (w tym porównanie z baseline'em). Przy wykresie dodaj przynajmniej jeden fragmentator np. z wyborem daty.
- Rozwiązanie umieść w repozytorium GitHub

Bonus (opcjonalny): wybierz 3 najlepiej sprzedające się produkty (wg średniej dziennej sprzedaży) i przygotuj dla nich osobne prognozy na 4 tygodnie.

1. Po pobraniu pliku csv z danymi otworzyłem go z wykorzystaniem Arkuszy Google i przeanalizowałem z grubsza rodzaje kolumn, które w nim występują.
2. Następnie wykorzystując bibliotekę pandas i prosty skrypt w pythonie, sprawdziłem podstawowe dane analityczne na temat danych z pliku.

Skrypt: `data_consistency_checker.py`

Ustaliłem, że plik nie posiada brakujących danych ani duplikatów. Dane zawierają informacje o sprzedaży od 1 stycznia 2022 do 30 stycznia 2024. Wartości w kolumnach oraz podstawowe miary statystyczne kolumn liczbowych nie wzbudziły podejrzeń co do błędnych wartości. Odchylenie standardowe w przypadku kolumny Inventory Level, a także duża różnica między wartością maksymalną (2267) a średnią (301,06) sugeruje, że poziom zapasów jest bardzo zróżnicowany. W przypadku Units Sold średnia 88,8 i mediana 84 są dość blisko, więc rozkład sprzedaży nie wydaje się mocno asymetryczny. Jeśli chodzi o Units Ordered mediana wynosi 0, ale średnia 89 i maksimum 1616. Oznacza to, że przez wiele dni

nie składano zamówień, natomiast okresowo pojawiały się bardzo duże zamówienia. W przypadku Price i Competitor Pricing zakresy, średnie i mediany są bardzo podobne, co jest sensowne, ponieważ gdyby różniły się zbyt mocno, mielibyśmy do czynienia z cenami dumpingowymi. W przypadku kolumny Discount zakres mówi nam, że maksymalna zniżka wynosi 25%, natomiast średnia kolumny Promotion, ponieważ jest to kolumna binarna przyjmująca wartości 0 lub 1, mówi nam, że promocją zostało objętych ok. 32,89% sprzedanych produktów, czyli niemal $\frac{1}{3}$. Tak samo jak w przypadku kolumny Promotion, średnia z kolumny Epidemic mówi nam, że w okresie epidemicznym miała miejsce sprzedaż równo 20% produktów. Jeśli chodzi o Demand, to wartości podstawowych miar statystycznych zawsze przyjmował większe niż Units Sold, więc popyt był trochę większy niż sprzedaż.

W zbiorze nie występują wartości ujemne, natomiast wartości zerowe są logiczne i uzasadnione.

Jeśli chodzi o zmienne kategoryczne, dane pochodzą z 5 sklepów i dotyczą sprzedaży 20 różnych produktów. Najwięcej kupowano artykułów spożywczych (30400), a najmniej, ponad 3 razy mniej niż artykułów spożywczych, elektroniki (9120). Warunki pogodowe miały spory wpływ na sprzedaż, najwięcej transakcji miało miejsce w pochmurne (24360) i słoneczne (22980) dni, natomiast aż około 50% mniej w stosunku do nich sprzedano w dni, kiedy padał śnieg. W dni deszczowe sprzedaż również była mniejsza. Jeśli chodzi o wpływ pór roku, to najwięcej sprzedaży miało miejsce w zimę, ale ogólnie ilość transakcji w różnych porach roku, jest do siebie zbliżona.

3. Po wstępnej analizie danych, zagregowałem dane sumując po sklepach i produktach, otrzymując łączną dzienną sprzedaż. Ze względu na agregację, od razu można założyć, że do modelu nie trafią zmienne takie jak: region (ponieważ zsumowana została sprzedaż ze sklepów znajdujących się w różnych regionach), kategoria (ponieważ zsumowana została sprzedaż produktów należących do różnych kategorii). Poza zsumowaniem liczby sprzedanych produktów we wszystkich sklepach, dokonałem następujących agregacji:

- Inventory Level - suma, czyli łączny stan magazynowy wszystkich produktów dla wszystkich sklepów
- Units Ordered - suma, czyli łączna liczba zamówionych sztuk produktów
- Demand - suma, czyli łączny popyt
- Price - średnia, średnia cena produktów sprzedanych danego dnia
- Competitor Price - średnia, średnia cena produktów sprzedanych danego dnia u konkurencji
- Discount - średnia, średnia wysokość rabatu udzielona na produkty sprzedane danego dnia
- Promotion - średnia, określi procent produktów objętych rabatem

- Epidemic - pierwsza wartość, ponieważ dane epidemiologiczne są takie same dla poszczególnych dni
- Seasonality - pierwsza wartość, ponieważ dla poszczególnych dni są takie same pory roku
- Weather Condition - procent udziałów, czyli jaka część transakcji miała miejsce w danych warunkach atmosferycznych

Skrypt: aggregate.py

4. Po zagregowaniu danych przystąpiłem do analizy i wyboru danych do modelu prognostycznego.

Na początku odrzuciłem zmienne reprezentujące czynniki wewnętrzne, ponieważ model ma uwzględniać jedynie czynniki zewnętrzne. A więc odrzuciłem:

- Inventory Level - zapas produktu zależy od firmy
- Units Ordered - firma zamawia produkt
- Discount - zniżki ustala firma
- Promotion - j.w.
- Price - cenę ustala firma

Następnie oceniłem, dla których zmiennych reprezentujących czynniki zewnętrzne, znane będą wartości na najbliższe 4 tygodnie. Były to:

- Date
- Seasonality

Przyszłe wartości reszty zmiennych nie są znane. Wartości te będę zatem szacował na podstawie danych historycznych.

W dalszej analizie uwzględniłem następujące potencjalne czynniki zewnętrzne: Demand, Competitor Pricing, Epidemic, Seasonality oraz zagregowane informacje o warunkach pogodowych. Dla zmiennych liczbowych obliczyłem współczynnik korelacji Pearsona względem Units Sold, natomiast dla zmiennych kategoriowych porównałem średnią dzienną sprzedaż pomiędzy poszczególnymi kategoriami.

Skrypt: external_factors_analysis.py

Czynnik	Korelacja z Units Sold	Ocena
Demand	+0.9813	ekstremalnie silna
Competitor Pricing	+0.9052	bardzo silna
Epidemic	-0.8978	bardzo silna ujemna
Weather_Sunny	+0.1750	słaba
Weather_Snowy	-0.1199	słaba
Weather_Rainy	-0.1021	słaba
Weather_Cloudy	+0.0433	praktycznie brak

Najsilniejszą dodatnią zależność zaobserwowałem dla zmiennej Demand, dla której współczynnik korelacji ze sprzedażą wyniósł 0,9813. Bardzo silną dodatnią korelację wykazała również zmienna Competitor Pricing (0,9052). Wyniki wskazują, że obie zmienne mogą dostarczać istotnych informacji podczas prognozowania sprzedaży.

Zmienne pogodowe wykazały znacznie słabszą zależność ze sprzedażą. Najwyższą bezwzględną korelację uzyskano dla Weather_Sunny (0,1750), natomiast dla Weather_Snowy, Weather_Rainy oraz Weather_Cloudy wartości wyniosły odpowiednio -0,1199, -0,1021 i 0,0433. Ze względu na słabą zależność ze sprzedażą oraz brak wiarygodnej informacji o pogodzie dla całego 28-dniowego horyzontu prognozy zdecydowałem się nie uwzględniać zmiennych pogodowych w modelu.

Zmienna Epidemic wykazała bardzo silną ujemną zależność ze sprzedażą. Współczynnik korelacji wyniósł -0,8978, a średnia dzienna sprzedaż spadła z 9581,92 szt. dla Epidemic = 0 do 6085,97 szt. dla Epidemic = 1.

Seasonality zostawiłem jako czynnik zewnętrzny. Analiza wykazała różnice w średniej dziennej sprzedaży pomiędzy porami roku: 9575,79 szt. latem, 8846,26 jesienią, 8777,05 zimą oraz 8346,36 wiosną. Dodatkową zaletą tej zmiennej jest pełna dostępność jej przyszłych wartości, ponieważ porę roku można jednoznacznie określić na podstawie daty, dlatego jej wykorzystanie nie wymaga prognozowania dodatkowej zmiennej ani nie powoduje ryzyka wycieku informacji.

Ostatecznie do dalszego modelowania wybrałem następujące czynniki zewnętrzne:

- Demand - bardzo silna zależność ze sprzedażą.
- Competitor Pricing - bardzo silna zależność ze sprzedażą.
- Seasonality - widoczne różnice sprzedaży pomiędzy porami roku, a przyszła wartość jest znana na podstawie daty.
- Epidemic - bardzo silna, odwrotna zależność ze sprzedażą.

5. Po wybraniu czynników zewnętrznych przeprowadziłem feature engineering, żeby stworzyć cechy związane ze sprzedażą.

Z daty utworzyłem cechy:

- DayOfWeek
- DayOfMonth, Month
- WeekOfYear
- IsWeekend.
-

Pozwolą one modelowi rozpoznawać m.in. różnice między dniami tygodnia, efekt weekendu oraz zmiany sezonowe w obrębie miesiąca i roku.

Dla zmiennej Units Sold utworzyłem opóźnienia (lagi) o 1, 7, 14, 21 i 28 dni, dzięki czemu model będzie mógł wykorzystywać historyczne wartości sprzedaży.

Dodatkowo obliczyłem średnie kroczące oraz odchylenia standardowe dla okresów 7, 14 i 28 dni. Przy ich obliczaniu zastosowałem przesunięcie o jeden dzień, aby cechy zawierały wyłącznie informacje dostępne przed prognozowanym dniem i nie powodowały wycieku danych.

Ze względu na najdłuższe, 28-dniowe okno historyczne pierwsze 28 obserwacji nie posiadało kompletu utworzonych cech i zostało usuniętych. W rezultacie liczba obserwacji zmniejszyła się z 760 do 732.

Skrypt: feature_engineering.py

6. Następnie podzieliłem dane na zbiór treningowy i walidacyjny. Zgodnie z treścią zadania, w zbiorze walidacyjnym znalazło się ostatnie 28 dni ze zbioru danych.

Skrypt: split_data.py

7. W celu ograniczenia liczby zmiennych wejściowych przeprowadziłem selekcję cech metodą Boruta, wykorzystując wyłącznie zbiór treningowy, aby uniknąć wycieku informacji ze zbioru walidacyjnego. Metoda porównuje istotność

rzeczywistych cech z losowo wygenerowanymi zmiennymi referencyjnymi, dzięki czemu pozwala wskazać cechy rzeczywiście istotne dla prognozowania sprzedaży. Algorytm wybrał następujące zmienne:

- czynniki zewnętrzne: Demand, Competitor Pricing, Epidemic,
- cechy historyczne sprzedaży: Lag 7, Lag 28, Rolling Mean 7 oraz Rolling Mean 14.

Odrzucone zostały pozostałe lags i statystyki kroczące, cechy kalendarzowe, zmienne opisujące pogodę oraz zakodowana zmienna Seasonality. Algorytm nie określił żadnej cechy jako niepewnej.

Wyniki potwierdziły wcześniejszą analizę czynników zewnętrznych, szczególnie duże znaczenie mają Demand, Competitor Pricing oraz Epidemic. Jednocześnie selekcja wskazuje na znaczenie tygodniowej i dłuższej sezonowości sprzedaży, reprezentowanej m.in. przez Lag_7 i Lag_28. Do dalszego modelowania przyjąłem cechy zaakceptowane przez algorytm Boruta.

Skrypt: `feature_selection.py`

8. Po selekcji cech, przygotowałem baseline. Jako punkt odniesienia dla właściwego modelu zastosowałem Seasonal Naive z sezonowością 7 dni. Metoda ta zakłada występowanie tygodniowej powtarzalności sprzedaży i wykorzystuje ostatni znany tydzień jako prognozę kolejnych tygodni. Baseline został wybrany ze względu na prostotę, brak konieczności trenowania modelu oraz uwzględnienie tygodniowego charakteru szeregu. Prognoza została wykonana dla całego 28-dniowego okresu walidacyjnego na podstawie danych dostępnych przed jego rozpoczęciem, dzięki czemu nie wykorzystuje informacji z przyszłości.

Dla modelu bazowego otrzymałem następujące wyniki:

Metryka	Wynik	Interpretacja
MAE	2595,36	Prognoza różniła się od rzeczywistej sprzedaży średnio o ok. 2595 sztuk dziennie.
RMSE	2945,89	Błąd uwzględniający silniejszą karę za duże pomyłki wyniósł ok. 2946 sztuk. Wyższa wartość RMSE od MAE wskazuje na występowanie dni ze szczególnie dużymi błędami prognozy.
MAPE	28,49%	Prognozy odbiegały od rzeczywistej sprzedaży średnio o około 28,5%.
Bias	-2539,79	Ujemny bias oznacza, że baseline miał wyraźną tendencję do zaniżania sprzedaży, średnio o ok. 2540 sztuk dziennie.

Ze względu na bardzo słabe wyniki uzyskane przez ten baseline, zdecydowałem się na inny. Wybrałem taki, w którym wartość dla danego dnia = sprzedaż z analogicznego dnia rok wcześniej. Metryki, chociaż nadal dały raczej średnie wyniki, poprawiły się.

Metryka	Wynik	Interpretacja
MAE	1513.61	Prognoza różniła się od rzeczywistej sprzedaży średnio o ok. 1514 sztuk dziennie.
RMSE	1951.90	Błąd uwzględniający silniejszą karę za duże pomyłki wyniósł ok. 1952 sztuki.
MAPE	21%	Prognozy odbiegały od rzeczywistej sprzedaży średnio o około 21%.
Bias	887,61	Baseline miał wyraźną tendencję do zawyżania sprzedaży, średnio o ok. 888 sztuk dziennie.

Skrypt: baseline.py

9. Następnie przystąpiłem do trenowania modeli i wyboru najlepszego z nich. Wybór modelu przeprowadziłem wyłącznie na zbiorze treningowym, pozostawiając właściwy 28-dniowy zbiór walidacyjny nietknięty do końcowej oceny.

Do porównania wybrałem modele Random Forest, HistGradientBoosting oraz XGBoost, ponieważ dobrze sprawdzają się w zadaniach regresyjnych opartych na danych tabelarycznych i pozwalają modelować nieliniowe zależności oraz interakcje pomiędzy zmiennymi.

Zastosowałem walidację szeregu czasowego typu expanding window obejmującą 5 kolejnych okresów testowych po 28 dni, co odpowiada docelowemu horyzontowi prognozy. Prognozy w każdym okresie generowano rekurencyjnie, dzięki czemu cechy historyczne sprzedaży były wyznaczone na podstawie wcześniejszych predykcji, a nie rzeczywistych wartości z okresu testowego.

Najlepsze i najbardziej stabilne wyniki uzyskał Random Forest. Średnio dla pięciu okresów osiągnął około MAE = 188,2, RMSE = 240,5 oraz MAPE = 2,17%. Dla porównania średni MAE wyniósł około 203,9 dla HistGradientBoosting oraz 202,5 dla XGBoost. Random Forest osiągnął najniższy MAE w 4 z 5 badanych okresów. Na podstawie tych wyników został wybrany jako model do końcowej 28-dniowej prognozy na wydzielonym zbiorze walidacyjnym.

Należy zauważyć, że algorytm Boruta wykorzystuje model Random Forest do oceny istotności zmiennych, co może częściowo faworyzować ten sam typ modelu na etapie późniejszego porównania. Z tego względu ostateczny wybór Random Forest nie został dokonany na podstawie samej selekcji cech, lecz na podstawie wyników pięciu niezależnych okresów walidacji czasowej w obrębie zbioru treningowego.

Skrypt: model_selection.py

10. Po wyborze najlepszego modelu na podstawie walidacji czasowej, Random Forest wytrenowałem na całym zbiorze treningowym, wykorzystując wyłącznie cechy wybrane wcześniej przez algorytm Boruta. Następnie wygenerowałem rekurencyjną prognozę sprzedaży dla 28-dniowego zbioru walidacyjnego. Przy wyznaczaniu cech historycznych wykorzystałem wcześniejsze predykcje modelu, dzięki czemu rzeczywiste wartości Units Sold ze zbioru walidacyjnego nie były używane podczas prognozowania.

Model osiągnął $MAE = 218,83$, $RMSE = 265,20$, $MAPE = 2,67\%$ oraz $Bias = 33,76$. Oznacza to przeciętny bezwzględny błąd na poziomie około 219 sztuk dziennie oraz średni błąd procentowy wynoszący 2,67%. Niewielki dodatni Bias wskazuje na lekką tendencję modelu do zawyżania prognozowanej sprzedaży. Wyniki są również znacznie lepsze od przyjętego modelu bazowego.

Skrypt: random_forest_forecast.py

11. W celu prezentacji wyników przygotowałem, zgodnie z treścią zadania, interaktywną aplikację internetową z wykorzystaniem biblioteki Streamlit oraz wykresów Plotly. Aplikacja umożliwia wizualne porównanie prognoz modelu Random Forest z rzeczywistą sprzedażą oraz modelem bazowym Seasonal Naive. Prezentowane są również najważniejsze miary jakości prognozy: MAE, RMSE, MAPE i Bias, wraz z ich krótką interpretacją. Użytkownik może wybrać interesujący go zakres dat za pomocą fragmentatora, a także przejrzeć szczegółowe wartości prognozy dla poszczególnych dni.

Skrypt: app.py

BONUS:

Bonus – prognoza sprzedaży 3 najlepiej sprzedających się produktów

W ramach zadania dodatkowego rozszerzyłem przygotowane rozwiązanie o osobne prognozy dla 3 najlepiej sprzedających się produktów. Produkty wybrałem na podstawie średniej dziennej liczby sprzedanych sztuk w całym dostępnym okresie. Najwyższe wartości uzyskały P0007 (499,62 szt.), P0004 (496,88 szt.) oraz P0009 (495,75 szt.).

Skrypt: top_products.py

Dla każdego z wybranych produktów dane następnie zagregowałem analogicznie, jak w przypadku sprzedaży całkowitej. Dzięki temu dla każdego produktu otrzymano osobny szereg czasowy obejmujący 760 dni.

Skrypt: aggregate_products.py

Następnie przeprowadziłem feature engineering analogiczny do zastosowanego w zadaniu głównym. Do finalnych modeli wykorzystałem ten sam zestaw cech, który został wcześniej wybrany metodą Boruta dla całkowitej sprzedaży: Demand, Competitor Pricing, Epidemic, UnitsSold_Lag_7, UnitsSold_Lag_28, UnitsSold_RollingMean_7 oraz UnitsSold_RollingMean_14. Pozwoliło to zachować spójną metodologię między główną częścią zadania a częścią dodatkową.

Skrypt: feature_engineering.py

Po utworzeniu cech pierwsze 28 obserwacji, dla których nie była dostępna pełna historia, zostało usuniętych. Dla każdego produktu pozostałe 732 obserwacje podzieliłem chronologicznie na 704 dni zbioru treningowego oraz ostatnie 28 dni zbioru walidacyjnego.

Skrypt: split_data_products.py

Do prognozowania wykorzystałem Random Forest z takimi samymi parametrami jak w zadaniu głównym. Dla każdego z trzech produktów wytrenowałem oddzielny model, dzięki czemu każdy model uczył się charakterystyki sprzedaży konkretnego produktu. Prognoza na okres walidacyjny była wykonywana rekurencyjnie, przy przewidywaniu kolejnych dni cechy historyczne obliczano na podstawie dostępnej historii oraz wcześniejszych predykcji modelu, bez wykorzystywania rzeczywistej sprzedaży z kolejnych dni okresu walidacyjnego.

Dla każdego produktu wygenerowałem osobną 28-dniową prognozę oraz obliczyłem metryki MAE, RMSE, MAPE i Bias. Wyniki poszczególnych modeli zostały zapisane zarówno w osobnych plikach, jak i w zbiorczym zestawieniu umożliwiającym łatwe porównanie jakości prognoz pomiędzy produktami.

Skrypt: random_forest_products_forecast.py

Aplikację Streamlit rozszerzyłem o osobny widok „3 najlepsze produkty”, dostępny z menu obok głównego widoku całkowitej dziennej sprzedaży. W części bonusowej dodałem fragmentator pozwalający wybrać jeden, kilka lub wszystkie trzy produkty. Wybór jest uwzględniany na wykresach prognozy i danych historycznych oraz w

prezentowanych tabelach, dzięki czemu użytkownik może interaktywnie analizować wyniki dla wybranych produktów.

Skrypt: app.py